



Make optional<T&> trivially copyable

Document number:

[P3836R0](#)

Date:

2025-09-11

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

Nevin Liber <nliber@anl.gov>

GitHub Issue:

wg21.link/P3836/github

Source:

github.com/eisenwave/cpp-proposals/blob/master/src/trivially-copyable-optional-ref.cow



ABSTRACT: Possibly due to a wording oversight, optional<T&> is not guaranteed to be trivially copyable. We propose to fix this in C++26. Due to certain design questions that concern LEWG, this issue is not simply handled as an LWG issue.

§ Contents

- 1 Introduction
 - 1.1 Wording problem
- 2 Motivation
- 3 Design
- 4 Wording
- 5 References

§ 1. Introduction

Making operations (at least construction) of `optional<T&>` trivial is the design intent of [P2988R12]:



”

3.3 Trivial construction

Construction of `optional<T&>` should be trivial, because it is straightforward to implement, and `optional<T>` is trivial. Boost is not.

However, possibly due to a wording oversight, trivial construction is not actually guaranteed, nor is trivial copyability in general. This status quo is inconsistent with the design of `optional<T>` as a whole: the special members (copy constructor, destructor, etc.) of the primary template are all specified to be trivial if the relevant operations on `T` are trivial. Therefore, `optional<int*>` is required to be trivially copyable, unlike `optional<int&>`.

This issue should be fixed in C++26. A US national body comment by Argonne National Laboratory requesting a solution has been submitted.

§ 1.1. Wording problem

Intuitively, `optional<T&>` (see [P2988R12]) is a `T*`, wrapped `optional`. In fact, the synopsis as specified in ([optional.optional.ref]) is as follows:

```
template<class T>
class optional<T&> {
    [...]

public:
    // [optional.ref.ctor], constructors
    constexpr optional() noexcept = default;

    [...]

    constexpr ~optional() = default;

    [...]

private:
    T* val = nullptr; // exposition only
};
```

If the only non-static data member was `val`, `optional<T&>` would be trivially copyable according to [class.prop] definition of "class, trivially copyable". However, the implementation has the freedom to add extra non-static data members, so no trivial operation is strictly guaranteed.

§ 2. Motivation

optional<T&> should be guaranteed to be trivially copyable for the following reasons:



- It makes the design consistent with the primary template optional<T>.
- Trivial copyability is a useful property in this case. It allows storing optional<T&> in a `struct` whose bytes are copied between processes, uploaded from CPU to GPU (e.g. in CUDA), etc.
- Trivial copyability also makes optional<T&> an implicit-lifetime class type, which makes it possible to start its lifetime in a `union` more easily via assignment operator, among other benefits.

§ 3. Design

There are two plausible options to fix the issue of optional<T&>:

1. Simply specify optional<T&> to be trivially copyable.
2. Additionally constrain its layout so it *actually* becomes a wrapper for a T*.

We propose the second option because trivial copyability is much more useful when the layout of a type is somewhat constrained. This makes it possible to `bit_cast<T*>(optional_ref)` or to encode optional<T&> using `memcpy` with *some* guarantees regarding its layout.



NOTE: The layout assumptions could still be false for an implementation where the layout of "wrapper `structs`" and the types they wrap is different, but that concern is largely hypothetical.

§ 4. Wording

The wording changes are relative to [N5014].

Immediately following [optional.optional.ref.general] paragraph 1, append a paragraph as follows:



A class type optional<T&> is a trivially copyable class and standard-layout class ([class.prop]). A class type optional<T&> has no non-static data members other than `val` and no base class B for which `is_empty_v<B>` ([meta.unary.prop]) is `false`.

§ 5. References

[N5014] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-08-05)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf>

[P2988R12] *Steve Downey, Peter Sommerlad*. `std::optional<T&>` (2025-04-04)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2988r12.pdf>

